```
└─$ bandit -r /home/rohinish/Desktop/Software_Security/Vulnerable_code/unsafe_os.py
[main]  INFO    profile include tests: None
[main]  INFO    profile exclude tests: None
[main]  INFO    cli include tests: None
[main]  INFO    cli exclude tests: None
[main]  INFO    running on Python 3.13.11
Run started:2026-02-12 13:30:45.310446

Test results:
>> Issue: [B605:start_process_with_a_shell] Starting a process with a shell, possible injection detected, security issue.
   Severity: High   Confidence: High
   CWE: CWE-78 (https://cwe.mitre.org/data/definitions/78.html)
   More Info: https://bandit.readthedocs.io/en/1.7.10/plugins/b605_start_process_with_a_shell.html
   Location: /home/rohinish/Desktop/Software_Security/Vulnerable_code/unsafe_os.py:9:4
8
9           os.system(command)
10

--------------------------------------------------

Code scanned:
        Total lines of code: 6
        Total lines skipped (#nosec): 0

Run metrics:
        Total issues (by severity):
                Undefined: 0
                Low: 0
                Medium: 0
                High: 1
        Total issues (by confidence):
                Undefined: 0
                Low: 0
                Medium: 0
                High: 1
Files skipped (0):
```

```
└─$ bandit -r /home/rohinish/Desktop/Software_Security/Vulnerable_code/hardcoded.py
[main]  INFO    profile include tests: None
[main]  INFO    profile exclude tests: None
[main]  INFO    cli include tests: None
[main]  INFO    cli exclude tests: None
[main]  INFO    running on Python 3.13.11
Run started:2026-02-12 13:32:14.887632

Test results:
>> Issue: [B105:hardcoded_password_string] Possible hardcoded password: 'SuperSecretPassword123!'
   Severity: Low   Confidence: Medium
   CWE: CWE-259 (https://cwe.mitre.org/data/definitions/259.html)
   More Info: https://bandit.readthedocs.io/en/1.7.10/plugins/b105_hardcoded_password_string.html
   Location: /home/rohinish/Desktop/Software_Security/Vulnerable_code/hardcoded.py:3:21
2
3           ADMIN_PASSWORD = "SuperSecretPassword123!"
4

--------------------------------------------------

Code scanned:
        Total lines of code: 8
        Total lines skipped (#nosec): 0

Run metrics:
        Total issues (by severity):
                Undefined: 0
                Low: 1
                Medium: 0
                High: 0
        Total issues (by confidence):
                Undefined: 0
                Low: 0
                Medium: 1
                High: 0
Files skipped (0):
```

```
┌──(rohinish㉿kali)-[~/Desktop/Software_Security/Vulnerable_code]
└─$ clang buffer_overflow.c -o buffer
buffer_overflow.c:8:11: warning: expression result unused [-Wunused-value]
    8 |     name[16];
      |     ~~~~ ~~^
buffer_overflow.c:8:4: warning: array index 16 is past the end of the array (that has type 'char[16]') [-Warray-bounds]
    8 |     name[16];
      |     ^    ~~
buffer_overflow.c:5:6: note: array 'name' declared here
    5 |     char name[16];
      |     ^
2 warnings generated.

┌──(rohinish㉿kali)-[~/Desktop/Software_Security/Vulnerable_code]
└─$ clang format_string.c -o format
format_string.c:5:12: warning: format string is not a string literal (potentially insecure) [-Wformat-security]
    5 |     printf(user_input);
      |            ^~~~~~~~~~
format_string.c:5:12: note: treat the string as an argument to avoid this
    5 |     printf(user_input);
      |            ^
      |            "%s",
1 warning generated.
```

```python
import os

def ping_host_vulnerable(hostname):
    print(f"Pinging {hostname}...")


    command = "ping -c 1 " + hostname

    os.system(command)


ping_host_vulnerable("google.com; echo 'SYSTEM COMPROMISED'")
```

```python
def check_admin_access(password):

    ADMIN_PASSWORD = "SuperSecretPassword123!"

    if password == ADMIN_PASSWORD:
        print("Access Granted.")
    else:
        print("Access Denied.")


user_input = input("Enter admin password: ")
check_admin_access(user_input)
```

```python
import os
import secrets  # IMPROVEMENT 1: Needed for secure comparison
import sys

def check_admin_access_secure(password):
    # Retrieve password from environment
    stored_password = os.getenv("ADMIN_PASS")

    # IMPROVEMENT 2: Validate the environment setup
    if not stored_password:
        print("CRITICAL ERROR: 'ADMIN_PASS' is not set in the environment.")
        sys.exit(1) # Exit with error status

    # IMPROVEMENT 3: Prevent Timing Attacks
    # 'secrets.compare_digest' takes constant time regardless of match failure position.
    if secrets.compare_digest(password, stored_password):
        print("Access Granted.")
    else:
        print("Access Denied.")

if __name__ == "__main__":
    try:
        user_input = input("Enter admin password: ")
        check_admin_access_secure(user_input)
    except KeyboardInterrupt:
        print("\nOperation cancelled.")
```

```c
/* improved_format_string.c */
#include <stdio.h>

// IMPROVEMENT 1: Use 'const' to enforce that this function reads-only
void secure_logger(const char *user_input) {

    // IMPROVEMENT 2: Null pointer check
    if (user_input == NULL) {
        printf("Error: No input provided.\n");
        return;
    }

    // Secure: Explicitly using "%s" prevents format string interpretation
    printf("Log entry: %s\n", user_input);
}

int main() {
    // Even if the input contains "%x", it is treated as plain text
    secure_logger("Hello World %x %x");
    return 0;
}
```

```c
#include <stdio.h>
#include <string.h>
#include <ctype.h>

#define BUFFER_SIZE 16

int main(void) {
    char name[BUFFER_SIZE];

    printf("Enter your name: ");

    // IMPROVEMENT 1: Use fgets instead of scanf
    // fgets reads up to BUFFER_SIZE - 1 characters, guaranteeing null-termination.
    if (fgets(name, sizeof(name), stdin) != NULL) {

        // Remove the newline character that fgets saves
        name[strcspn(name, "\n")] = 0;

        // IMPROVEMENT 2: Input Sanitization
        // Verify the input contains only letters or spaces
        for (int i = 0; i < strlen(name); i++) {
            if (!isalpha(name[i]) && !isspace(name[i])) {
                printf("Error: Invalid characters in name.\n");
                return 1;
            }
        }

        printf("Hello %s\n", name);
    } else {
        printf("Error reading input.\n");
    }

    return 0;
}
```

```python
import subprocess
import shutil
import re

def ping_host_secure(hostname):

    if not re.match(r"^[a-zA-Z0-9.-]+$", hostname):
        print(f"Security Alert: Invalid hostname format '{hostname}'")
        return

    print(f"Pinging {hostname}...")

    if not shutil.which("ping"):
        print("Error: Ping utility not found.")
        return

    try:
        result = subprocess.run(
            ["ping", "-c", "1", hostname],
            check=True,
            timeout=5,
            capture_output=True,
            text=True
        )
        print("Ping successful!")

    except subprocess.TimeoutExpired:
        print("Error: Ping timed out.")
    except subprocess.CalledProcessError:
        print("Ping failed (Host unreachable).")
    except Exception as e:
        print(f"An error occurred: {e}")

# This will now be blocked by the Regex check
ping_host_secure("google.com; echo 'SYSTEM COMPROMISED'")

# This will work
ping_host_secure("google.com")
```

```c
#include <stdio.h>
#include <string.h>

int main(void){
     char name[16];
    printf("Enter your name:");
    scanf("%s",name);
   name[16];
      printf("Hello %s\n",name);
     return 0;
}
```

```c
#include <stdio.h>

void format(char *user_input) {

    printf(user_input);
}

int main() {
    format("Hello World %x %x ");
    return 0;
}
```